# News Title–Body Consistency Checker using LLM + RoBERTa + SBERT

*by*
Md Arbaz (2206289)
T. Saketh Ram (2206282)
Mani Kanta (2206281)
*under guidance of*
Dr. Bhaskar Mondal (Assistant Professor)

Department of Computer Science and Engineering
**National Institute of Technology Patna**

December 9, 2025

# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

# Introduction

- Misleading news titles distort facts and manipulate public perception.
- Our system evaluates whether a news title accurately reflects its body.
- Uses LLM summarization + SBERT similarity + RoBERTa fine-tuning + SentiBERT sentiment analysis.

# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

# Problem Statement

We aim to:

- Detect mismatches between news headlines and content.
- Build an automated system that verifies consistency.
- Reduce spread of misinformation by identifying misleading titles.

# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

# Objectives

- Clean Title + Body text.
- Summarize body using LLM.
- Generate embeddings using SBERT.
- Fine-tune RoBERTa for classification.
- Use SentiBERT for sentiment-based features.
- Train/Validation/Test split.
- Evaluate using F1, Precision, Recall, Confusion Matrix.

# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

# Dataset Overview

- Columns: **ID, Title, Body, Label**
- Labels: Consistent / Inconsistent
- Preprocessing:
  - HTML cleaning
  - Lowercasing
  - Stopword removal
  - Lemmatization

# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

# Text Cleaning Steps

- Remove tags, digits, special symbols.
- Normalize whitespace.
- Tokenize text.
- Remove stopwords.
- Body and title are cleaned separately.

# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

# LLM-Based Summarization

- News body is passed through a fine-tuned LLM for abstractive summarization.
- The summary retains the essential meaning while removing unnecessary details.
- Helps reduce noise such as repeated sentences, filler text, and irrelevant phrases.
- Produces cleaner and shorter input for downstream models.
- Leads to faster similarity computation and improved classification performance.

# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

# Sentiment Extraction using SentiBERT

- SentiBERT extracts sentiment signals from both the title and the generated summary.
- Sentiments considered: positive, negative, neutral, or mixed.
- Comparing title–summary sentiment helps detect emotional exaggeration.
- Mismatched sentiment often indicates misleading or sensationalized headlines.
- Sentiment features are later combined with SBERT and RoBERTa outputs.

# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

# Semantic Similarity with SBERT

- SBERT converts both title and summary into dense embeddings.
- Embeddings capture the deeper semantic meaning of each text.
- Cosine similarity measures how close or different they are:
  - High similarity $\rightarrow$ consistent title
  - Low similarity $\rightarrow$ possible misleading title
- SBERT provides a strong semantic feature for final prediction.
- Similarity score is combined with sentiment and RoBERTa outputs.

# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

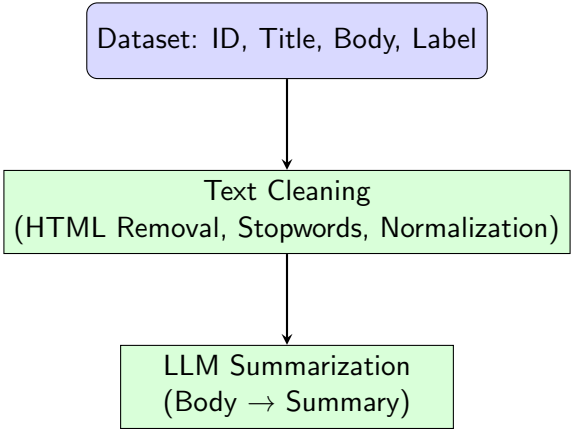
# RoBERTa Fine-Tuning

- RoBERTa is fine-tuned using (title, summary) input pairs.
- Model learns to detect contextual consistency between headline and article.
- Captures deeper patterns such as exaggeration, missing context, or distortion.
- Outputs high-quality classification probabilities (Real / Misleading).
- Works as the main component in the final prediction pipeline.

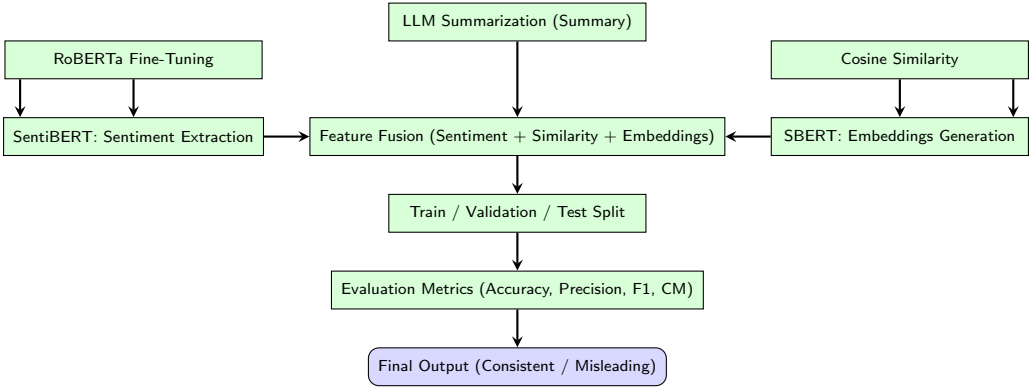
# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

# Workflow Diagram - Part 1



# Workflow Diagram - Part 2 (Ultra Compact Fit)



# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

# Dataset Split

- 70% Training
- 15% Validation
- 15% Test
- Balanced across labels

# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

# Evaluation Metrics

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix

# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

# Accuracy Results

```
Fold 5 → Accuracy: 0.8377

===== FINAL CROSS-VALIDATION RESULTS =====
Accuracy:  0.8645
Precision: 0.8921
Recall:    0.8463
F1 Score:  0.8680
```

Figure 1: *

Accuracy Value of the Model

# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

# Confusion Matrix

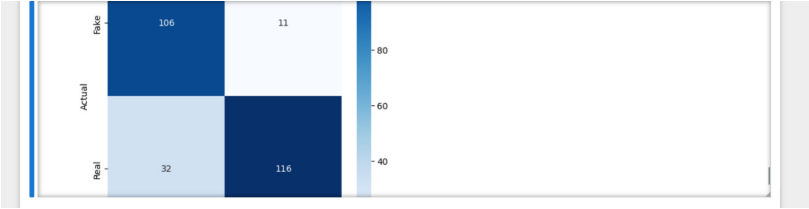



Figure 2: *

Confusion Matrix of the Model

# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

# Code Snapshot – Summarization

```python
numeric_ids = df["id"].astype(str).str.extract(r"(\d+)").dropna()
max_id = numeric_ids.astype(int)[0].max() if not numeric_ids.empty else 0
new_id = max_id + 1
for i, r in df.iterrows():
    if not str(r["id"]).strip():
        df.at[i, "id"] = str(new_id)
        new_id += 1

# Final cleaned dataset saved
df = df[["id", "clean_title", "clean_text", "label"]].rename(columns={"clean_title":"title","clean_text":"text"})
df.to_csv("cleaned_roberta_fixed_final.csv", index=False)
print("Cleaned rows:", len(df))

# ====================================================================
# ROBERTA-BASED SUMMARIZATION (we use BART: facebook/bart-large-cnn)
# ====================================================================
print("\nLoading RoBERTa-based summarizer (BART)...")
summ_model_name = "facebook/bart-large-cnn"
summ_tokenizer = AutoTokenizer.from_pretrained(summ_model_name)
summ_model = AutoModelForSeq2SeqLM.from_pretrained(summ_model_name)

tqdm.pandas(desc="Summarizing")
def summarize(text):
    inp = summ_tokenizer(text, return_tensors="pt", truncation=True, max_length=1024)
    ids = summ_model.generate(**inp, max_length=128, num_beams=4)
    return summ_tokenizer.decode(ids[0], skip_special_tokens=True)

df["summary"] = df["text"].progress_apply(summarize)
df["input_text"] = df["title"] + " " + df["summary"]
df.to_csv("summarized_roberta_fixed_final.csv", index=False)
print("Summarization complete.")
```

Figure 3: *

Implementation of the Model

# Code Snapshot – SBERT Similarity

```python
# ===== Use minimal TrainingArguments to be compatible with older transformers
training_args = TrainingArguments(
    output_dir=f"finbert_fold_{fold}",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=3,
    weight_decay=0.01,
    logging_dir=f"./logs/fold_{fold}",
    save_total_limit=1
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_ds,
    eval_dataset=val_ds,
    tokenizer=tokenizer,
    compute_metrics=compute_metrics
)

# train
trainer.train()

# explicit evaluation/prediction (works regardless of evaluation_strategy support)
preds = trainer.predict(val_ds)
y_true = preds.label_ids
y_pred = np.argmax(preds.predictions, axis=1)

accs.append(accuracy_score(y_true, y_pred))
precs.append(precision_score(y_true, y_pred))
```

Figure 4: *

Implementation of the Model

# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

# Final System Output

- Clean Title + Clean Body
- Summary from LLM
- Similarity Score
- Sentiment Comparison
- Final Label (Consistent / Misleading)

# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

# Conclusion

- Developed robust multi-model pipeline.
- Uses LLM + SBERT + RoBERTa + SentiBERT.
- Effectively detects misleading news titles.

# Table of Contents


1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Preprocessing
6. Summarization (LLM)
7. SentiBERT
8. SBERT Similarity
9. RoBERTa Fine-Tuning
10. Workflow Diagram
11. Train–Test–Validation Split
12. Evaluation Metrics
13. Accuracy Image
14. Confusion Matrix
15. Code Snapshots
16. Final System Output

# References


- Research papers on LLM summarization
- SBERT official documentation
- RoBERTa fine-tuning guide

# Thank You!